

Routing Table

Name	Endpoint	HTTP Method	Description
Home Page	/	GET	Home page, server responds by serving index.html.
Add Driver Page	/33343276/JinRuo/add-driver	GET	Server responds with add_driver.html which contains the add driver form.
Add Driver Page	/33343276/JinRuo/add-driver	POST	Server retrieves information from request and adds a new Driver object into the database, redirecting to list drivers page when Driver object is successfully added.
List Drivers Page	/33343276/JinRuo/list-drivers	GET	Server responds by serving list_drivers.html which lists all drivers' details in a table.
Add Package Page	/33343276/JinRuo/add-package	GET	Server responds with add_package.html which contains the add package form.
Add Package Page	/33343276/JinRuo/add-package	POST	Server retrieves information from request and adds a new Package object into the database, redirecting to list packages page when Package object is successfully added.
List All Packages Page	/33343276/JinRuo/list-packages	GET	Server responds by serving list_packages.html which lists all packages' details in a table.
Delete Driver by ID Page	/33343276/JinRuo/delete-driver	GET	Server responds with delete_driver.html which contains the delete driver form.
Delete Driver by ID Page	/33343276/JinRuo/delete-driver-req?driver_id=D31-33-PMY	GET	Deletes a driver by the given driver ID, in the query string, from the driver database.
Delete Package by ID Page	/33343276/JinRuo/delete-package	GET	Server responds with delete_package.html which contains the delete package form.
Delete Package by ID Page	/33343276/JinRuo/delete-package-req?package_id=PKL-JY-065	GET	Deletes a package by the given package ID, in the query string, from the database.
List Driver by Department Page	/33343276/JinRuo/list-drivers-dept	GET	Server responds with the select driver department form.
List Driver by Department Page	/33343276/JinRuo/list-drivers-dept	POST	Lists drivers' details of the selected department in a table.
List Drivers Page/ List Driver by Department Page (Delete Driver Button)	/33343276/JinRuo/delete-driver-req?driver_id=D31-33-PMY	POST	Deletes a driver by the driver ID of the specified row, in the query string, from the database, then redirects to list all drivers page.
List Packages Page (Delete Package Button)	/33343276/JinRuo/delete-package-req?package_id=PKL-JY-065	POST	Deletes a driver by the driver ID of the specified row, in the query string, from the database, then redirects to list all drivers page.
Sign Up Page	/33343276/JinRuo/users/signup	GET	Server responds by serving signup.html containing the sign up form.
Sign Up Page	/33343276/JinRuo/users/signup	POST	Adds new user details to firestore database if the username is not added before and the details meet the conditions. If successful, the user is redirected to login page, else the application renders the invalid_data.html.
Log In Page	/33343276/JinRuo/users/login	GET	Server responds by serving login.html containing the login form.
Log In Page	/33343276/JinRuo/users/login	POST	Check if entered details is a valid user in the database, redirects to home page if successful, renders invalid_data.html if not.
Sign Up API Endpoint	/33343276/JinRuo/api/v1/users/signup	POST	Uses the details given in the request body to add a new user to the database, returns a status message indicating a success/fail
Log In API Endpoint	/33343276/JinRuo/api/v1/users/login	POST	Uses the details given in the request body to check if the user exists in the database to log in, returns a status message indicating a success/fail

Add Driver API Endpoint	/33343276/JinRuo/api/v1/drivers/add	POST	Uses the details given in the request body to add a new driver to the database, returns a json object containing the mongoDB id and driver_id of the new driver if successfully added.
List All Drivers API Endpoint	/33343276/JinRuo/api/v1/drivers	GET	Lists all driver details in json object list with the details of their assigned packages.
Delete Driver by ID API Endpoint	/33343276/JinRuo/api/v1/drivers/delete?driver_id=12345678	DELETE	Uses the query string driver_id (mongoDB id) given to delete the driver and its assigned packages from the database, returns json object with acknowledgement of delete and deleted items number.
Update Driver by ID API Endpoint	/33343276/JinRuo/api/v1/drivers/update	PUT	Uses the details given in the request body to update the driver using the driver mongoDB id indicated in the request body with the department and driver license also listed in the body in a json object, returns a status message indicating a success/fail
Add Package API Endpoint	/33343276/JinRuo/api/v1/packages/add	POST	Uses the details given in the request body to add a new package to the database, returns a json object containing the mongoDB id and package_id of the new package if successfully added.
List All Packages API Endpoint	/33343276/JinRuo/api/v1/packages	GET	Lists all package details in json object list with the details of their assigned driver.
Delete Package by ID API Endpoint	/33343276/JinRuo/api/v1/packages/delete/123456789	DELETE	Uses the route parameter (package mongoDB id) given to delete the package from the database and also delete from the assigned driver's assigned packages list, returns json object with acknowledgement of delete and deleted items number.
Update Package by ID API Endpoint	/33343276/JinRuo/api/v1/packages/update	PUT	Uses the details given in the request body to update the package using the package mongoDB id indicated in the request body with the new package destination also listed in the body in a json object, returns a status message indicating a success/fail
CRUD Count Stats Page	/33343276/JinRuo/stats	GET	Server responds by serving stats.html to show all current CRUD operations count by using page or using API endpoints.
404	*	GET	Server responds by serving 404.html when a non-existent endpoint is queried.

Steps to run application locally:

1. Initialise workspace using command **npm init -y** in terminal.
2. Install all required modules using command **npm install ejs bootstrap express firebase-admin mongoose** in terminal.

3. Start the application by using command **node app.js**

*Note: Application was configured to run on 2 virtual machines, to run locally, change line 73 of app.js file as shown below before running app:

```
73  const url = 'mongodb://10.192.0.4:27017/assignment2';
```

Change the url to 'mongodb://localhost:27017/assignment2'

4. Open **http://localhost:8080/** in browser to start using the application.

5. Only the homepage is accessible without a valid user login, please signup/login using the navbar or by clicking any page which is not the home page when not logged in yet.
6. Sign Up
Create an alphanumeric username of 6 characters and a password between 5-10 characters for sign up. No username can be duplicated in the users database, any same username will fail to sign up. Skip this step if already signed up before.
7. Log In
After successfully signing up, it will redirect to login page. Login with created details, will fail if no credentials match the users in database. Users successfully logged in will be redirected to the home page. Then, the features can be accessed.
8. Start using application features
9. Add a new driver into the system
Press the `Add Driver` button in the navigation bar or use <http://localhost:8080/33343276/JinRuo/add-driver> to locate the Add New Driver Form, where all form fields must be filled out to add a valid driver. If any attributes are invalid, the application will serve the Invalid Data page.
10. List all drivers and their attributes in a table
Press the `List Drivers` button in the navigation bar or use <http://localhost:8080/33343276/JinRuo/list-drivers> to locate the page where all drivers and their attributes will be shown in a table. There is also a delete button for every driver, which can be used to quickly delete the driver from the database. After successful deletion, the list all drivers page will be shown.
11. List all drivers of a certain department and their attributes in a table
Press the `List Drivers/Department` button in the navigation bar or use <http://localhost:8080/33343276/JinRuo/list-drivers-dept> to locate the page showing the form to select a department of drivers the user wants to list. After pressing the select button, the table containing all drivers of the selected department, and their attributes will be shown in the table.
12. Delete a driver by their unique driver id
Press the `Delete Driver` button in the navigation bar or use <http://localhost:8080/33343276/JinRuo/delete-driver> to locate the page showing the form to provide a driver id to delete the specified driver. If the submitted

driver id is the id of a valid driver in the database, the driver will be deleted, and the page will show the list of all drivers after deletion. If the driver id is invalid, the application will serve the Invalid Data page.

13. Add a new package into the system

Press the `Add Package` button in the navigation bar or use <http://localhost:8080/33343276/JinRuo/add-package> to locate the Add New Package Form, where all form fields must be filled out to add a valid package, excluding the description which is optional. If any attributes are invalid, the application will serve the Invalid Data page.

14. List all packages and their attributes in a table

Press the `List Packages` button in the navigation bar or use <http://localhost:8080/33343276/JinRuo/list-packages> to locate the page where all packages and their attributes will be shown in a table. There is also a delete button for every package, which can be used to quickly delete the package from the database. After successful deletion, the list all packages page will be shown.

15. Delete a package by their unique package id

Press the `Delete Package` button in the navigation bar or use <http://localhost:8080/33343276/JinRuo/delete-package> to locate the page showing the form to provide a package id to delete the specified package. If the submitted package id is the id of a valid package in the database, the package will be deleted, and the page will show the list of all packages after deletion. If the package id is invalid, the application will serve the Invalid Data page.

16. Show CRUD operation statistics

Press the `Stats` button in the navigation bar or use <http://localhost:8080/33343276/JinRuo/stats> to locate the page showing the number of CRUD operations done in the application whether by API endpoints or by the actual page of the application.

17. Sign Up via API endpoint

Use <http://localhost:8080/33343276/JinRuo/api/v1/users/signup> to sign up by sending the sign up details in the body of the request in a json object.

```
1  {
2    ... "username": "joeyyy",
3    ... "password": "123456",
4    ... "confirmPassword": "123456"
5  }
6
```

18. Log In via API endpoint

Use <http://localhost:8080/33343276/JinRuo/api/v1/users/login> to login by sending the login details in the body of the request in a json object.

```
1 {
2   ... "username": "joeyyy",
3   ... "password": "123456"
4 }
5
```

19. Add a new driver via API endpoint

Use <http://localhost:8080/33343276/JinRuo/api/v1/drivers/add> to add a new driver by sending the driver attributes in a json object in the body of the request.

```
{
  "driver_name": "Alex",
  "driver_department": "Food",
  "driver_licence": "ABC12",
  "driver_isActive": true,
}
```

20. Get all drivers in database via API endpoint

Use <http://localhost:8080/33343276/JinRuo/api/v1/drivers> to get a list of json objects containing all the drivers data and the data of the packages assigned to them.

21. Delete driver by id via API endpoint

Use

http://localhost:8080/33343276/JinRuo/api/v1/drivers/delete?driver_id=12345678 to delete the driver by its MongoDB id given as a query string from the database, the packages of that driver will also be deleted.

22. Update driver by id via API endpoint

Use <http://localhost:8080/33343276/JinRuo/api/v1/drivers/update> to update the department and driver license of the driver using the attributes sent in the body of the request in a json object. The driver will be identified using the id (MongoDB id) also listed in the request body json object.

```
{
  "id": "66cda5ab54d2030bb551d6e2",
  "driver_licence": "XYZ12",
  "driver_department": "Food"
}
```

23. Add a new package via API endpoint

Use <http://localhost:8080/33343276/JinRuo/api/v1/packages/add> to add a new package by sending the package attributes in a json object in the body of the request.

```
{
  "package_title": "PixelPhones",
  "package_weight": 1.25,
  "package_destination": "Melbourne",
  "isAllocated": true,
  "driver_id": "66cda5ab54d2030bb551d6e2",
}
```

24. Get all packages in database via API endpoint

Use <http://localhost:8080/33343276/JinRuo/api/v1/packages> to get a list of json objects containing all the packages data and the data of the drivers assigned to them.

25. Delete package by id via API endpoint

Use <http://localhost:8080/33343276/JinRuo/api/v1/packages/delete/1234567> to delete the driver by its MongoDB id given as a route parameter from the database, the package will also be removed from the assigned driver's assigned packages list.

26. Update package by id via API endpoint

Use <http://localhost:8080/33343276/JinRuo/api/v1/packages/update> update the package destination using the attributes sent in the body of the request in a json object. The package will be identified using the package_id (MongoDB id) also listed in the request body json object.

```
{
  "package_id": "66cd9f08a1e79b53efc2c2d2",
  "package_destination": "Perth",
}
```

Steps to deploy on 1 VM:

1. Create a GCP Platform Virtual Machine instance.
2. Open terminal
3. Install git

```
sudo apt update
```

```
sudo apt install -y git
```

4. Install node.js and npm

```
1 # installs nvm (Node Version Manager)
2 curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.0/install.sh | bash
3
4 # download and install Node.js (you may need to restart the terminal)
5 nvm install 20
6
7 # verifies the right Node.js version is in the environment
8 node -v # should print `v20.16.0`
9
10 # verifies the right npm version is in the environment
11 npm -v # should print `10.8.1`
```

5. Install MongoDB using commands listed below:

```
sudo apt-get install gnupg curl
```

```
curl -fsSL https://pgp.mongodb.com/server-7.0.asc | sudo gpg -o
/usr/share/keyrings/mongodb-server-7.0.gpg --dearmor
```

```
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-
7.0.gpg ] https://repo.mongodb.org/apt/ubuntu focal/mongodb-org/7.0
multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
```

```
sudo apt-get update
```

```
sudo apt-get install -y mongodb-org
```

```
sudo systemctl enable mongod
```

```
sudo systemctl start mongod
```

6. Open port 8080 for application.
7. Clone github repository to VM instance using:
git clone (https link to clone git repository)
Enter authentication details for git.
8. Install the project's dependencies, build components, and run the server.

```
cd project_folder
npm install
node app.js
```

9. Keep the terminal alive, go to the GCP instance page, and copy the external IP address. Open your browser and navigate to <http://externalIP:8080>
*Note: Change all urls used to start with <http://externalIP:8080> instead of localhost

Steps to deploy on 2 VMs (1 hosting the app & 1 hosting MongoDB database):

1. Create 2 GCP Platform Virtual Machine instances.
2. On instance 1, complete steps 2-4 in the previous section.
3. On instance 2, complete step 5.
4. Stay on instance 2, we need to allow MongoDB server to accept requests from anywhere, update the /etc/mongod.conf file, as shown below:

sudo nano /etc/mongod.conf

Change line: bindIp: 127.0.0.1

To

bindIp: 0.0.0.0

5. Open port 8080 for application.
6. Change line 73 of app.js file to match your VM2 internal IP address.

```
73  const url = 'mongodb://10.192.0.4:27017/assignment2';
```

Change the url to 'mongodb://**yourVM2InternalIP**:27017/assignment2'

The internal IP used in given code was used to test on my machine.

7. Complete steps 7-10 on VM instance 1. Use externalIP of VM instance 1.